

Análisis de Algoritmos y Algoritmos Fundamentales

Temas Principales

- Análisis de Algoritmos.
- Ecuaciones de Recurrencia.
- Algoritmos Fundamentales.

Formato de Entrega

Se deberá completar la actividad en el Aula Virtual, agregando una carpeta compartida con todos los archivos fuente correspondiente por ejercicio.

La carpeta deberá llamarse Apellido_Nombre_TPN_3.

Ejercicios Obligatorios a Entregar.....	2
Ejercicio 1.....	2
Ejercicio 2.....	3
Ejercicio 3.....	3
Ejercicio 4.....	4
Ejercicio 5.....	4
Ejercicios de Práctica.....	5



Ejercicios Obligatorios a Entregar

Dado los siguientes programas, se pide obtener mediante las medidas asintóticas desarrolladas en clase, el $T(n)$, e indicar su tipo de complejidad.

Cada solución debe estar acompañada de su desarrollo Matemático.

Ejercicio 1

```
import java.util.Scanner;

public class NumerosPrimos {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingrese el valor de MAX: ");
        int MAX = scanner.nextInt();

        int i = 1;
        for (; i <= MAX; ) {
            int cD = 0;
            int j = 1;
            for (; j <= i; ) {
                if (i % j == 0) {
                    cD++;
                }
                j++;
            }
            if (cD == 2) {
                System.out.println(i + " es primo!");
            }
            i++;
        }
        scanner.close();
    }
}
```

Ejercicio 2

```
import java.util.Scanner;
public class ConversionBinaria {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingrese el valor de N: ");
        int N = scanner.nextInt();
        int[] arreglo = new int[50];
        int num, i, j, k;
        for (k = 1; k <= N; k++) {
            num = k;
            i = 0;
            System.out.print(k + ": ");
            while (num > 0) {
                arreglo[i++] = num % 2;
                num /= 2;
            }
            for (j = i - 1; j >= 0; j--) {
                System.out.print(arreglo[j]);
            }
            System.out.println();
        }

        scanner.close();
    }
}
```

Dada las siguientes funciones, obtener mediante las relaciones de recurrencia la medida asintótica el $T(n)$, e indicar su tipo de complejidad.

Cada solución debe estar acompañada de su desarrollo Matemático.

Ejercicio 3

```
public static int fRecu(int n){
    if(n <= 1){
        return n * 6 + 1;
    }else{
        int suma = 2 * fRecu(n / 3) + fRecu(n / 3);
        return suma + fRecu(n / 3) + fRecu(n / 3) ;
    }
}
```

Ejercicio 4

```
public static int fRecu(int n){  
    if(n < 1){  
        return 4 + n - 2;  
    }else{  
        int valor = 4 * fRecu(n - 3);  
        return fRecu(n - 3) + valor + fRecu(n - 3) ;  
    }  
}
```

Ejercicio 5

```
public static int Func (int n){  
    if(n < 1)  
        return 1;  
    else{  
        int inicio=0, aux = n;  
        while( inicio < n )  
            aux = aux + (inicio++) * (n-1);  
        return Func(n - 1) + aux;  
    }  
}
```



Ejercicios de Práctica

Ejercicio 1

```
public static int fRecu(int n){
    if(n <= 1){
        return n * 6 + 1;
    }else{
        int suma = 2 * fRecu(n / 2) ;
        return suma + fRecu(n / 2) ;
    }
}
```

Ejercicio 2

```
public static int fRecu(int n){
    if(n <= 1){
        return n * 6 + 1;
    }else{
        return 2 + fRecu(n / 2) ;
    }
}
```

Ejercicio 3

```
public static int fRecu(int n){
    if(n <= 1){
        return n * 2;
    }else{
        return 2 + fRecu(n - 2) ;
    }
}
```

Ejercicio 4

Dado el algoritmo de búsqueda binaria, obtener su valor de $T(n)$